Escalations — Backend Integration Guide

Oct 8, 2026 · @Sonali

Purpose and status

The escalation flow is fully built in the frontend but stores everything in each browser's localStorage. Connecting it means building one FastAPI router plus storage, then rewiring one frontend file: `src/lib/escalation-store.tsx`.

Already working (frontend only):

- Escalations tab: project folders, escalation matrix, people, open-day limits per category and per checkpoint, with inheritance.
- Simulate on a project: works out each open item's level and notifies Developer, Manager or CTO.
- Developer: Fixed, then Waiting on others with Nudge (once a day) and Undo.
- Manager: Approve (item becomes compliant, with an approval record as evidence) or Reject (reason required, clock restarts).
- CTO: alerts on the Home page.
- Manager Home: All projects, PIT Armour, SAATHI and MPS tabs.

Mocked today, needs the backend:

Area	Today	Needed
Escalation state	localStorage key `audit-intelligence-escalations-v1`, one browser only	Shared database, same data for every user
Open items	Seeded demo items (PIT Armour 5, SAATHI 2, MPS 2)	Created when a checkpoint fails (from the existing verifiers)
Escalation run	Only when someone clicks Simulate	Simulate plus a daily scheduled run
Notifications	In-app list only	In-app plus email (test mailboxes during the demo)

Area	Today	Needed
Approval evidence	Text record (who, when, what)	Plus a PNG from the screenshot agent
Who is who	Role picked with View as	Real user, role and projects from login
SAATHI / MPS manager data	Mock copies of PIT Armour	Real per-project data

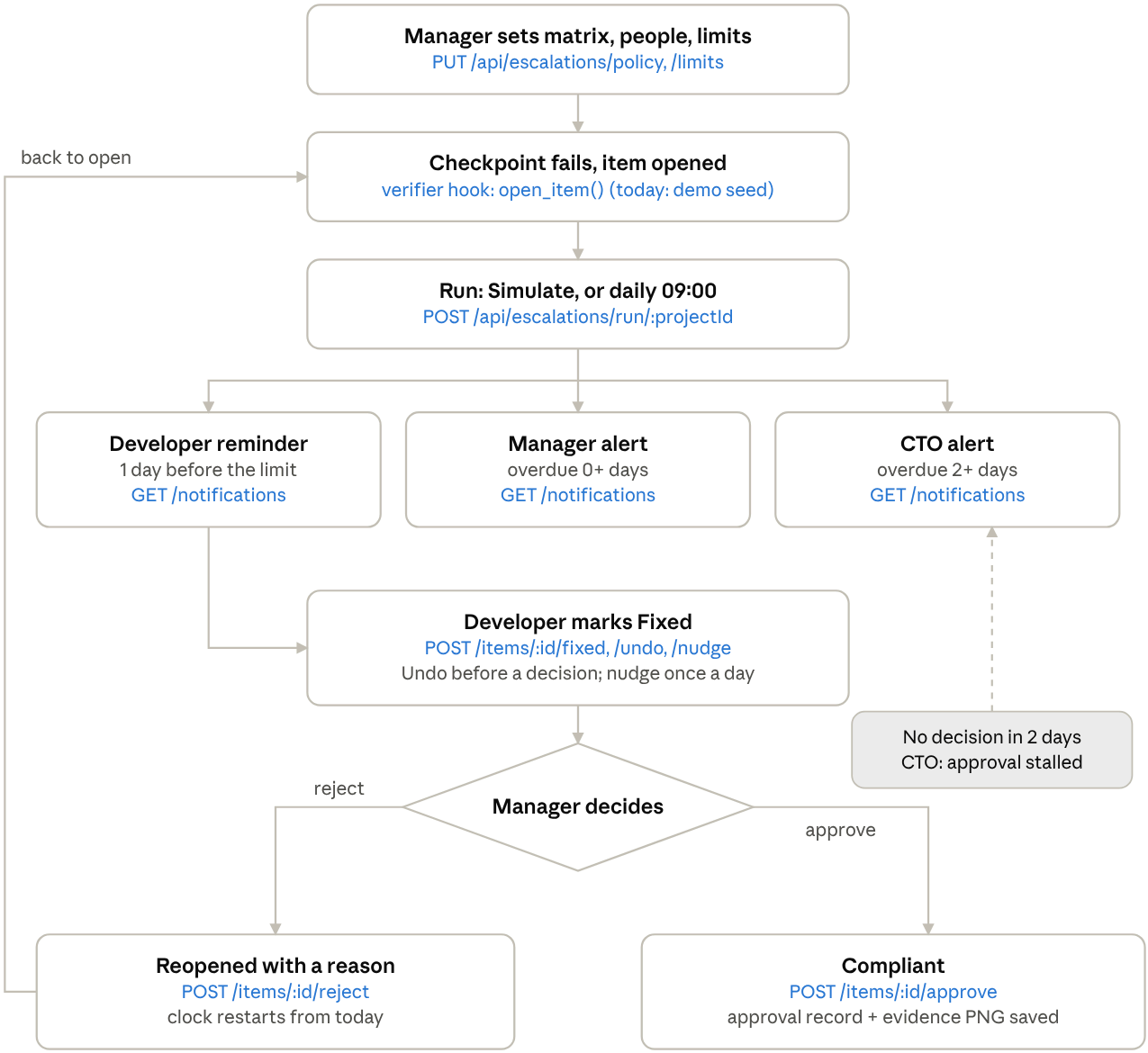
Done means: two people in two browsers see the same items, a Fixed in one shows as waiting for approval in the other, and approving stores evidence on the server.

End-to-end flow

An item keeps escalating on every run until the Manager approves its fix; a rejection sends it back to the start with the clock reset.

A failing checkpoint climbs to the CTO until its fix is approved

Blue lines = backend endpoint to build



escalation flow · setup to compliant, with backend endpoints

The CTO is reached two ways: an item 2+ days overdue, or a fix the Manager has left undecided for 2+ days. All day counts come from the project's matrix in the Escalations tab.

Architecture

Every screen reads and writes escalations through one React context, so the backend work lands in one new FastAPI router on the backend and one store file on the frontend. No screen component needs to change.

Frontend (TanStack Start + React, port 8081 locally):

Piece	Path	Role
API client	`src/lib/api-client.ts`	`apiFetch()` / `apiFetchOrNull()` against `VITE_API_BASE_URL` (default `http://localhost:8000`). Add the escalation calls here.
Escalation store	`src/lib/escalation-store.tsx`	`EscalationProvider` and `useEscalations()`. Today it holds state in React and saves to localStorage. **Swap the action bodies for API calls.**
Types and rules	`src/lib/escalation-data.ts`	Types that mirror the backend records; `resolveLimit()` and `ageItem()` rules the backend must copy exactly.
Audit store	`src/lib/audit-store.tsx`	Checkpoints (mock `initialCheckpoints`). `resolveCheckpoint()` is called on Approve to mark the checkpoint compliant.
Role and login	`src/lib/role-store.tsx`, `src/lib/auth-store.tsx`	Role from the View as picker (`audit-intelligence-view-role`); login is a demo session (`audit-intelligence-session`).

Backend (FastAPI, `backend/app`, port 8000):

Piece	Path	Reuse for escalations
App and CORS	`app/main.py`	Register the new router. **CORS allows only GET and POST today; add PUT and DELETE.**
Settings	`app/settings.py`	`frontend_origin`, Microsoft Graph credentials, `ms_test_mailbox`, `evidence_dir`.
Routers	`app/routers/*.py`	Pattern to copy: `APIRouter(prefix="/api/...")`, Pydantic models in `app/models.py`.
JSON file store	`app/services/invites.py`	`_load()` / `_save()` on a JSON file: fine for the POC; use a database for production.
Email	`app/services/invites.py` (`_send_invite_email`)	Graph `sendMail` already works. Reuse it for escalation emails.
Evidence PNG	`app/clients/evidence_screenshot.py`	Playwright `_render_and_save()`: reuse it to render the approval record as a PNG.

The frontend must send its local origin: start the backend with `FRONTEND_ORIGIN=http://localhost:8081` (the default is 8080).

Data model

Five records cover the whole flow. Field names below are the ones in `src/lib/escalation-data.ts`; keep them (camelCase in JSON, via Pydantic aliases as the other routers do) so the frontend types need no change.

1. EscalationPolicy (one per project)

Field	Type	Default	Meaning
projectId	string		`project-a` PIT Armour, `project-b` SAATHI, `project-c` MPS

Field	Type	Default	Meaning
reminderBeforeDays	int	1	Developer reminder this many days before the limit
managerAfterDays	int	0	Manager notified this many days after the limit
ctoAfterDays	int	2	CTO notified this many days after the limit
approvalLimitDays	int	2	Manager must approve a fix within this, else CTO notified
people.developer / manager / cto	{name, email}	Rahul Verma / Neha Gupta / CTO	Who gets each level; email blank = no email

2. Limits (one per project)

Field	Type	Meaning
defaultLimitDays	int or null	Project-wide default; null = org default (2 days)
categoryLimits	map category → int	Override for a whole category
checkpointLimits	map checkpointId → int	Override for one checkpoint

3. OpenItem (one per failing checkpoint per project)

Field	Type	Meaning
id	string	Today `esc-<checkpointId>`; any unique id works
projectId, checkpointId	string	Which checkpoint, in which project
owner	string	Developer who owns the fix
openedAt	ISO datetime	When it failed; reset on Reject

Field	Type	Meaning
status	`open`, `pending-approval`, `compliant`	Lifecycle
fixedAt, fixedBy, overdueDaysAtFix	optional	Set on Fixed
approvedAt, approvedBy	optional	Set on Approve
rejectReason	optional string	Set on Reject, shown to the developer
nudges	list of ISO datetimes	One per nudge; cleared on Fixed, Undo, Reject
approval	ApprovalRecord, optional	Evidence, set on Approve
history	list of {at, text}	Audit trail shown in the UI

4. Notification

Field	Type	Meaning
id	string	Unique
runAt	ISO datetime	Run time for escalations; action time for others
projectId, itemId	string	What it is about
recipient	`developer`, `manager`, `cto`	Who sees it
kind	`escalation`, `approval-stalled`, `nudge`, `fixed`	Type
title, detail	string	Text shown, e.g. "#15 Code review…" and "Open 4 days · limit 2 days · 2 days overdue"

5. RunSummary (latest per project) and ApprovalRecord

- RunSummary: `runAt`, `projectId`, `counts` {developer, manager, cto}, `notifications`, `emailNote`.

- ApprovalRecord: `checkpointRef`, `checkpointName`, `projectName`, `fixedBy`, `fixedAt`, `overdueDaysAtFix`, `nudges` (count), `approvedBy`, `approvedAt`. Add `evidenceFile` for the PNG when the screenshot agent is wired.

API contract

Thirteen endpoints replace the store's localStorage. Each item action returns the **updated OpenItem** plus any **new notifications**, so the frontend can update in place without a reload.

#	Method and path	Request body	Response	Replaces store action
1	`GET /api/escalations/config/{projectId}`		`{policy, defaultLimitDays, categoryLimits, checkpointLimits}` (defaults if never saved)	`getConfig`
2	`PUT /api/escalations/policy/{projectId}`	`Policy` (whole matrix + people)	saved config	`setPolicy`, `setPerson` (the Save button)
3	`PUT /api/escalations/limits/{projectId}`	`{defaultLimitDays?, categoryLimits?, checkpointLimits?}`; `null` value = remove override	saved config	`setDefaultLimit`, `setCategoryLimit`, `setCheckpointLimit`
4	`DELETE /api/escalations/config/{projectId}`		defaults	`resetProject`
5	`GET /api/escalations/items?projectId=`		`OpenItem[]` (omit projectId = all the user's projects)	`items`
6	`POST /api/escalations/run/{projectId}`		`RunSummary`	`runEscalation` (Simulate)
7	`GET /api/escalations/runs/latest`		`{projectId: RunSummary}`	`lastRun`
8	`POST /api/escalations/items/{id}/fixed`		`{item, notifications}`	`markFixed`
9	`POST /api/escalations/items/{id}/undo`		`{item}`	`withdrawFix`
10	`POST /api/escalations/items/{id}/nudge`		`{item, notifications}`; 429 if nudged in the last 24 h	`nudge` (`canNudge` stays client-side)
11	`POST /api/escalations/items/{id}/approve`	`{by}`	`{item, notifications}` incl. `item.approval`	`approve`

#	Method and path	Request body	Response	Replaces store action
12	`POST /api/escalations/items/{id}/reject`	`{by, reason}` (reason required, 422 if blank)	`{item, notifications}`	`reject`
13	`GET /api/escalations/notifications?role=&projectId=&since=`		`Notification[]` newest first	`notifications`, `useRoleNotifications`

Demo only: `POST /api/escalations/demo/reset` re-seeds the 9 demo items (replaces `resetDemo`).

Errors: 404 unknown item; 409 when the action does not fit the status (Fixed on a non-open item, Approve on a non-pending item); 422 bad body. The frontend shows the message from `apiFetch`'s thrown error.

Example: Simulate on SAATHI

```
POST /api/escalations/run/project-b
→ {
  "runAt": "2026-10-08T05:36:00Z",
  "projectId": "project-b",
  "counts": {"developer": 2, "manager": 2, "cto": 1},
  "notifications": [
    {"id": "n-1", "runAt": "2026-10-08T05:36:00Z", "projectId": "project-b",
"itemId": "esc-cp-25",
     "recipient": "cto", "kind": "escalation", "title": "#25 Security
vulnerabilities are tracked to closure",
     "detail": "Open 5 days · limit 2 days · 3 days overdue · owner Vivek
Pillai · Manager Neha Gupta"}
  ],
  "emailNote": "Email off: no recipient emails set in the Escalations tab."
}
```

Business rules

The backend must reproduce `resolveLimit()` and `ageItem()` from `src/lib/escalation-data.ts` exactly, or the counts on screen will not match the emails. Port them to Python line for line.

1. **Limit for an item** (first match wins): checkpoint override → category override → project default → org default of 2 days.
2. **Days open** = whole days since `openedAt`, rounded down, never below 0.
3. **Overdue days** = days open − limit (negative = days left).

4. **Level** of an open item, checked in this order:
   a. overdue ⩾ `ctoAfterDays` → **cto**
   b. overdue ⩾ `managerAfterDays` → **manager**
   c. days left ⩽ `reminderBeforeDays` → **developer**
   d. otherwise no level, nobody is notified
5. **Who a run notifies:** level developer → developer only. Level manager → developer and manager. Level cto → developer, manager and CTO. Every run notifies again; there is no "already sent" suppression.
6. **Stalled approval:** a `pending-approval` item whose fix has waited ⩾ `approvalLimitDays` → one `approval-stalled` notification to the CTO on each run.
7. **Fixed:** only from `open`. Sets `fixedAt`, `fixedBy`, `overdueDaysAtFix` = max(0, overdue), clears nudges and `rejectReason`. Notifies the manager (`kind: fixed`).
8. **Undo:** only from `pending-approval`, before the Manager decides. Back to `open` with the **original** `openedAt` (the clock does not restart).
9. **Nudge:** only on `pending-approval`, at most once per 24 hours per item. Notifies the manager (`kind: nudge`).
10. **Approve:** only from `pending-approval`. Status `compliant`, store the ApprovalRecord, notify the developer, and mark the checkpoint compliant (today `resolveCheckpoint(id, "compliant", "Approved", …)` in the audit store).
11. **Reject:** only from `pending-approval`; reason required. Back to `open`, `openedAt` = now (the clock **restarts**), store `rejectReason`, notify the developer.
12. **Every change** appends a line to `history` with a timestamp. This is the audit trail.

Worked example (PIT Armour seed, defaults): #15 opened 4 days ago, limit 2 → 2 days overdue ⩾ CTO's 2 → notifies developer, manager and CTO. #8 opened 2 days ago → 0 overdue ⩾ manager's 0 → developer and manager. #19 opened 1 day ago → 1 day left ⩽ 1 → developer only. #21 matches #15. #6 (fixed 3 days ago, still pending) is a stalled approval → CTO. A Simulate on a fresh PIT Armour seed therefore gives Developer 4, Manager 3, CTO 3 notifications; use this as a backend unit test.

Frontend wiring points

Almost all frontend work is in `src/lib/escalation-store.tsx`, plus new functions in `src/lib/api-client.ts`. Screens call `useEscalations()` and keep working unchanged.

#	File and place	Today	Change to
1	`src/lib/api-client.ts` (end of file)	No escalation calls	Add `getEscalationConfig`, `saveEscalationPolicy`, `saveEscalationLimits`, `resetEscalationConfig`, `listEscalationItems`, `runEscalation`, `getLatestRuns`, `markItemFixed`, `undoItemFix`, `nudgeItem`, `approveItem`, `rejectItem`, `listNotifications`, each with `apiFetch`
2	`escalation-store.tsx`, first `useEffect` (load)	Reads localStorage, seeds demo items, adds missing SAATHI/MPS seed	Fetch items, configs (3 projects), latest runs and notifications from the API; drop the seed and migration
3	`escalation-store.tsx`, second `useEffect` (save) and the `storage` listener	Writes the whole state to localStorage; syncs tabs	Delete both. Poll `GET /items` and `/notifications` every 30 s, or on window focus, to see other users' changes
4	`updateConfig` / `setPolicy` / `setPerson`	Merge into local config	Call `PUT /policy` or `PUT /limits`, then put the response into state
5	`runEscalation(pid)`	Computes levels in the browser, returns `RunSummary` synchronously	`await POST /run/{pid}`; **becomes async**. Update `src/components/audit/CopilotSimulate.tsx` `simulate()` to `await` it
6	`markFixed`, `withdrawFix`, `nudge`, `approve`, `reject`	`updateItem()` changes local state	Call the endpoint; replace the item with the returned one; prepend returned notifications
7	`approve()`, last line	`resolveCheckpoint(...)` in the browser audit store	Backend marks the checkpoint compliant; keep the local call so the UI updates at once, or refetch checkpoints
8	`resetDemo`	`setState(fresh())`	`POST /demo/reset`, then reload; hide the button outside the demo
9	`ready` flag	True once localStorage loads	True once the first fetch returns; set an `error` flag on failure and show it
10	Who is acting (`by` in approve/reject, the manager name)	`MANAGER_PERSONA.name` (Neha Gupta), role from View as	The logged-in user from the backend session

Keep client-side: `ageOf()` (labels like "2 days overdue" between runs), `canNudge()` (button state; the server still enforces), `meta()` (checkpoint names).

Screens that read the store (no change needed): `routes/escalations.tsx`, `components/developer/EscalatedWork.tsx`, `components/developer/DevSections.tsx`, `components/developer/DeveloperDashboard.tsx`, `components/manager/EscalationPanels.tsx`, `components/manager/ManagerPortfolio.tsx`, `components/manager/ManagerDashboard.tsx`, `routes/index.tsx` (CTO alerts).

Separate from escalations: the Manager tabs for SAATHI and MPS use mock data built in `src/lib/manager-data.ts` (`MANAGER_PROJECTS`, `variant()`, `rebrand()`). Real per-project health, releases, team and decisions need their own endpoints later.

Email, scheduler, evidence, auth and item creation

These six pieces have no frontend work beyond what is listed; they make the flow real instead of demo-driven.

1. Where open items come from. Today they are seeded. In production, create an OpenItem when a checkpoint turns non-compliant: hook the existing verifiers (`app/services/cp10_verify.py`, `cp38_verify.py`, `cr_compliance_verify.py`) so a failing result calls `open_item(projectId, checkpointId, owner)`. If an item for that checkpoint is already open, leave it alone. A passing result on an open item can close it, or wait for the Manager's approval; decide which.

2. Email. Reuse the Graph `sendMail` code in `app/services/invites.py` (`_send_invite_email`). Send one email per recipient per run, listing all of their items, not one per item. Use the emails in the policy's `people`; if blank, send nothing and say so in `emailNote`. **During the demo, send only to `ms_test_mailbox`, never to real people.** Set `emailNote` to what actually happened (sent to N, or why not).

3. Scheduler. Simulate stays as a manual trigger. Add a daily run per project, for example 09:00 IST, that calls the same function as `POST /run`. Options: APScheduler inside FastAPI, or an external cron hitting the endpoint. Store each run so `GET /runs/latest` works.

4. Approval evidence (PNG). On Approve, render the ApprovalRecord as an HTML card and save a PNG with `_render_and_save()` from `app/clients/evidence_screenshot.py`, into `evidence_dir`. Store the file name on the record (`evidenceFile`) and serve it with a `GET /api/escalations/items/{id}/evidence` endpoint returning `{dataUrl}`, as the CP10/CP38 evidence endpoints already do. The frontend card (`ApprovalRecordCard` in `components/manager/EscalationPanels.tsx`) can then show an image link.

5. Auth and roles. Today anyone can act as any role via View as. The backend must check:

Action	Allowed for
Fixed, Undo, Nudge	The item's owner (developer)
Approve, Reject	The project's manager

Action	Allowed for
Edit matrix and people	Manager or compliance admin
Edit open-day limits	Compliance admin (read-only for Manager in the UI)
Simulate, demo reset	Admin, Manager (demo only)
Read alerts	Each role sees only its own notifications

The Manager "All projects" view needs `GET /api/me` returning the user's name, role and project ids, so it shows only the projects that manager owns. Today it shows all three.

6. Invites (already built). Invite links (`app/routers/invites.py`) lock the viewer to the Developer view for one project. Escalation endpoints should accept that invite token as a developer identity for that project.

Build order, acceptance tests and AI prompt

Build in this order; after step 4 the demo already works across browsers.

1. **Storage + config:** models, JSON or database store, endpoints 1–4. Wire `getConfig` / `setPolicy` / limits in the store. Add PUT and DELETE to CORS.
2. **Items + rules:** port `resolveLimit` and `ageItem`, seed the 9 demo items, endpoint 5 and demo reset.
3. **Run:** endpoints 6, 7, 13. Make `runEscalation` async in the store and `CopilotSimulate.tsx`.
4. **Item actions:** endpoints 8–12 and the frontend calls. Delete the localStorage code.
5. **Evidence PNG** on Approve.
6. **Email** to the test mailbox, then the **daily scheduler**.
7. **Auth** (`/api/me`, role checks) and **item creation** from the verifiers.

Acceptance tests

- [ ] Fresh seed, Simulate PIT Armour → Developer 4, Manager 3, CTO 3.
- [ ] Set a checkpoint limit of 5 days on #15 → it drops out of Manager and CTO on the next run.
- [ ] Browser A marks #15 Fixed → browser B (Manager) sees it in Fixes waiting within 30 s.
- [ ] Nudge twice in a row → second call returns 429; the button shows "again tomorrow".

- [ ] Undo keeps the original opened date; Reject restarts it and shows the reason to the developer.
- [ ] Approve → item compliant, checkpoint compliant, approval record (and PNG) stored.
- [ ] Approve on an open item → 409.
- [ ] A fix pending 3 days with approval limit 2 → CTO gets "approval stalled" on the next run.
- [ ] Manager All projects shows the same counts as the three project tabs added up.
- [ ] No email reaches a real person during the demo.

Ready-to-paste prompt for an AI coding assistant (run it in the repo root):

```
You are connecting the Escalations feature of the Audit Intelligence
dashboard to its FastAPI backend.

Context:
- Frontend: TanStack Start + React in src/. All escalation state is in
src/lib/escalation-store.tsx
  (EscalationProvider / useEscalations), currently saved to localStorage key
  "audit-intelligence-escalations-v1". Types and rules are in
src/lib/escalation-data.ts.
  HTTP helpers are in src/lib/api-client.ts (apiFetch, VITE_API_BASE_URL).
- Backend: FastAPI in backend/app. Copy the router pattern of
app/routers/invites.py and the
  JSON-file store of app/services/invites.py. Register the router in
app/main.py and add PUT and
  DELETE to the CORS allow_methods.

Task:
1. Backend: add app/routers/escalations.py (prefix /api/escalations),
app/services/escalations.py
   and Pydantic models with camelCase aliases matching escalation-data.ts
(Policy, ProjectConfig,
   OpenItem, Notification, RunSummary, ApprovalRecord). Endpoints:
   GET/DELETE /config/{projectId}, PUT /policy/{projectId}, PUT
/limits/{projectId},
   GET /items?projectId=, POST /run/{projectId}, GET /runs/latest,
   POST /items/{id}/fixed|undo|nudge|approve|reject, GET /notifications?
role=&projectId=&since=,
   POST /demo/reset. Item actions return {item, notifications}.
2. Port resolveLimit() and ageItem() to Python exactly, and the run, fixed,
undo, nudge, approve
   and reject logic from escalation-store.tsx, including history lines.
Rules: nudge once per
```

```
   24 h (else 429); actions in the wrong status return 409; reject needs a
reason (else 422) and
   resets openedAt; undo keeps openedAt.
3. Frontend: in escalation-store.tsx replace localStorage load/save and the
storage listener with
   API fetches (poll items and notifications every 30 s and on focus). Make
each action call its
   endpoint and merge the response. runEscalation becomes async: update
   src/components/audit/CopilotSimulate.tsx to await it. Keep the
useEscalations() interface the same
   so no screen changes.
4. Do not send email to real people. Email, scheduler, evidence PNG and auth
come later.
5. Test: a fresh seed + Simulate PIT Armour gives Developer 4, Manager 3, CTO
3 notifications;
   two browsers see the same items. Run `npx tsc --noEmit` and fix new type
errors.
```

The full flow and data model are also in the PRD: Audit Intelligence PRD. The changed frontend files and their paths are in `Desktop\Frontend_Timepass\Escalation_Handover`.